

Reviewer Pack — Matroid Rank Complexity of Monotone DNF for k-CLIQUE

Entry 1c08aaba24ea · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 03:33:39 UTC

Matroid Rank Complexity of Monotone DNF for k-CLIQUE

Entry ID: 1c08aaba24ea **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 03:33:39 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone Circuit Complexity

Statement:

There exists a matroid M on the vertex set of the complete graph K_n such that (i) the rank function $r(M)$ is submodular under the conjunction of DNF terms, (ii) for any monotone DNF formula of size $\text{poly}(n)$, $r(M)$ is $O(\log n)$, and (iii) for the k -CLIQUE indicator function, $r(M)$ is $\Omega(n)$.

Rationale (proposer's reasoning):

The rank function of a matroid is inherently submodular, which aligns with requirement (i). By associating each term in the DNF with a subset in the matroid, the rank could measure the complexity of the DNF. For functions with small DNF size, the rank would be logarithmic, but for k -CLIQUE, which requires exponentially many terms, the rank would scale linearly with n . This connects matroid theory to circuit complexity, offering a new angle for lower bounds.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 073e77f367b73a44

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n, k):
        if n < k:
            return None
        clique = set(range(k))
        for i in range(k, n):
            if all(i not in edge for edge in itertools.combinations(clique, 2)):
                clique.add(i)
        return clique

    def is_submodular(ranks):
        n = len(ranks)
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    if ranks[i] + ranks[j] < ranks[k]:
                        return False
        return True

    def matroid_rank(k_clique):
        # Simple heuristic to simulate a matroid rank function
        return len(k_clique)

    n = random.randint(5, 40)
    k = random.randint(2, min(n - 1, 5))
    k_clique = generate_k_clique(n, k)

    if k_clique is None:
        return {
            "metric_name": "matroid_rank",
            "metric_value": 0,
            "instances_tested": 1,
```

```

        "conjecture_holds": False,
        "counterexample": "k-clique not possible for n, k"
    }

    ranks = [matroid_rank(k_clique)]
    for _ in range(29):
        new_k_clique = generate_k_clique(n, k)
        if new_k_clique is None:
            continue
        ranks.append(matroid_rank(new_k_clique))

    rank_function_submodular = is_submodular(ranks)
    average_rank = sum(ranks) / len(ranks)
    conjecture_holds = rank_function_submodular and (average_rank <= 2 * math.log(n)) and (average_rank > 0.8)

    return {
        "metric_name": "matroid_rank",
        "metric_value": average_rank,
        "instances_tested": len(ranks),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "rank_function_not_submodular_or_out_of_bounds"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"rank_function_not_submodular_or_out_of_bounds\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

c_name': 'matroid_rank', 'metric_value': 25.0, 'instances_tested': 30, 'conjecture_holds': False, 'c
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 23.0, 'instances_tested': 30, 'conjecture_hol
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 9.0, 'instances_tested': 30, 'conjecture_hol
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 38.0, 'instances_tested': 30, 'conjecture_hol
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 16.0, 'instances_tested': 30, 'conjecture_hol
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 36.0, 'instances_tested': 30, 'conjecture_hol

```

```

TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 19.0, 'instances_tested': 30, 'conjecture_holds': True}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 17.0, 'instances_tested': 30, 'conjecture_holds': True}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7eea0a2f.py", line 93, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7eea0a2f.py", line 93, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with KeyError 'seed' before producing valid results, preventing reliable evaluation of conjecture support/falsification. | next: Fix test code to handle seed tracking and re-run experiments with proper error handling

11. Audit log (LLM calls)

Total LLM calls: 13

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	61001
2	propose	ollama_remote	qwen3:8b	0	0	118580
3	propose	ollama_remote	qwen3:8b	0	0	47590
4	propose	ollama_remote	qwen3:8b	0	0	81479
5	propose	ollama_remote	qwen3:8b	0	0	89862
6	preregistration	ollama_remote	qwen3:8b	0	0	27775
7	novelty	ollama_remote	qwen3:8b	0	0	24202
8	novelty	ollama_remote	qwen3:8b	0	0	22194
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15295
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7156
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7549
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8760
13	judge	ollama_remote	qwen3:8b	0	0	18237

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 529680 ms total latency. Provider mix: {'ollama_remote': 13}

(full prompt+response transcripts available in `research/audit/1c08aaba24ea.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1c08aaba24ea.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1c08aaba24ea.tar.gz` (if generated)